Reviewer Pack — Minimal Order of Groupoid Categorical Dimensions and Frege P…

Entry f75cec009dec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 08:44:42 UTC

Minimal Order of Groupoid Categorical Dimensions and Frege Proof Depth

Entry ID: `f75cec009dec` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 08:44:42 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Groupoids) **Field B** (complexity object): Complexity Theory (Frege Proof Complexity)

Statement:

> For every instance φ of size n with polynomially bounded groupoid categorical dimension d(φ) and a corresponding Frege proof tree T(φ), the depth d(T(φ)) of the proof tree is upper-bounded by O(d(φ)^2 log n).

Rationale (proposer's reasoning):

> Groupoids provide a categorial framework that may capture complex dependencies in proof structures. A lower bound on the groupoid categorical dimension could imply structural complexity constraints on the Frege proof depth, potentially leading to new insights into proof complexity.

Taxonomy category: `category_theory_to_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `153877b74ebe6113`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated instances φ with polynomially bounded groupoid categorical dimension d(φ) ≤ 40 and corresponding Frege proof tree T(φ), the depth d(T(φ)) satisfies O(d(φ)^2 log n). The criterion is falsified if there exists an instance where the ratio of d(T(φ)) to d(φ)^2 log n exceeds 1.0 for any seed or if the correlation coefficient is less than 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "groupoid categorical dimension" AND "Frege proof complexity" - "proof tree depth" IN Category Theory AND Complexity Theory - "O(d(φ)^2 log n) bound" IN groupoids AND Frege proofs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0]*p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```
    def frege_proof_depth(proof_tree):
        if not proof_tree:
            return 0
        return 1 + max(frege_proof_depth(child) for child in proof_tree)

    def groupoid_categorical_dimension(instance):
        # Placeholder function to compute the dimension of a groupoid instance
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 40)

    n = random.choice([5, 10, 15, 20, 30, 40])
    d_phi = groupoid_categorical_dimension(n)
    proof_tree = [[], [], []]  # Placeholder for the Frege proof tree
    d_T_phi = frege_proof_depth(proof_tree)

    if d_phi == 0:
        return {
            "metric_name": "d(T(φ)) / d(φ)^2 log n",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    ratio = Fraction(d_T_phi, d_phi**2 * math.log(n))
    conjecture_holds = ratio <= 1.0

    return {
        "metric_name": "d(T(φ)) / d(φ)^2 log n",
        "metric_value": float(ratio),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Ratio {ratio} exceeds 1.0"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results)
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)
        counterexample = next(result["counterexample"] for result in results if result["conjecture
```

```
        RESULT = f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first
    print(RESULT)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a6cefd9.py", line 91, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a6cefd9.py", line 73, in run_trial
    ratio = Fraction(d_T_phi, d_phi**2 * math.log(n))
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to a TypeError, which prevented the computation of the required metrics. The critic challenged the result, and without unambiguous support from the test, we cannot confirm SUPPORTED. | next: Investigate the cause of the TypeError and attempt to run the test again with proper error handling.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14553
2	preregistration	ollama_remote	glm4:latest	0	0	18680
3	novelty	ollama_remote	glm4:latest	0	0	8413
4	novelty	ollama_remote	glm4:latest	0	0	9165
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36279
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8453
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8230
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10511
9	judge	ollama_remote	glm4:latest	0	0	16676

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 130961 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f75cec009dec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f75cec009dec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f75cec009dec.tar.gz` (if generated)